```
        status = 0
if epsilon > 0.1:                                    17
    epsilon -= (1/epochs)
```

- ***1*** Creates a list to store loss values so we can plot the trend later
- ***2*** The main training loop
- ***3*** For each epoch, we start a new game.
- ***4*** After we create the game, we extract the state information and add a small amount of noise.
- ***5*** Converts the numpy array into a PyTorch tensor and then into a PyTorch variable
- ***6*** Uses the status variable to keep track of whether or not the game is still in progress
- ***7*** While this game is still in progress, plays to completion and then starts a new epoch
- ***8*** Runs the Q-network forward to get its predicted Q values for all the actions
- ***9*** Selects an action using the epsilon-greedy method
- ***10*** Translates the numerical action into one of the action characters that our Gridworld game expects
- ***11*** After selecting an action using the epsilon-greedy method, takes the action
- ***12*** After making a move, gets the new state of the game
- ***13*** Finds the maximum Q value predicted from the new state
- ***14*** Calculates the target Q value
- ***15*** Creates a copy of the qval array and then updates the one element corresponding to the action taken
- ***16*** If reward is –1, the game hasn't been won or lost and is still in progress
- ***17*** Decrements the epsilon value each epoch

Note

Why did we add noise to the game state? It helps prevent "dead neurons," which can happen with the use of rectified linear units (ReLU) as our activation function. Basically, because most of the elements in our game state array are 0s, they won't play nice with ReLU, which is technically nondifferentiable at 0. Hence, we add a tiny bit of noise so that none of the